

1. Fase de Datos: Construir el Dataset Sintético Inteligente

- **Diseñar la Base de Datos:** Implementen una base de datos relacional ligera en SQLite o PostgreSQL. Deben crear exactamente las 5 tablas clave requeridas por el reto: *Siniestros, Pólizas, Asegurados, Beneficiarios/Proveedores y Documentos*.
- **Injectar los Casos de Prueba (Truco Ganador):** No generen datos completamente aleatorios que no simulen patrones reales. Inyecten manualmente registros específicos diseñados para activar las reglas críticas (RF-01 a RF-07). Por ejemplo, agreguen un siniestro por robo ocurrido dentro de las primeras 24 horas de vigencia de la póliza o descripciones textuales con narrativas casi idénticas. Esto les servirá para superar con éxito las "Pruebas de Fuego" en vivo ante el jurado.

2. Fase de Desarrollo: El Core del Motor Híbrido

- **Módulo de Reglas Determinísticas:** Programen el archivo `src/rules/fraud_rules.py` utilizando Python para evaluar las reglas del negocio de la RF-01 a la RF-07. Si un siniestro coincide con la lista restrictiva o tiene adulteración documental evidente, debe sumar puntos de riesgo de forma directa e inmediata.
- **Módulo Estadístico y NLP (Filtro Probabilístico):** Desarrollen `src/models/fraud_model.py` implementando un algoritmo de Machine Learning para identificar anomalías numéricas en montos y frecuencias de reclamos. Paralelamente, creen un script de Procesamiento de Lenguaje Natural (NLP) que use embeddings de texto o similitud de coseno para comparar las descripciones y detectar narrativas clonadas con más del 85% de similitud.
- **Consolidación del Score (0-100):** Diseñen la lógica matemática que combine los puntajes de las Reglas, el ML y el NLP. El resultado final debe traducirse estrictamente en los rangos del semáforo sugerido: Verde (0-40), Amarillo (41-75) o Rojo (76-100).

3. Fase de Inteligencia Artificial: El Agente y la Explicabilidad

- **Configurar el Agente LLM:** En el archivo `src/ai_agent/claims_agent.py`, integren la API de Claude o ChatGPT. Este agente recibirá el JSON de un siniestro ya calificado y redactará automáticamente una justificación textual clara del porqué de la alerta.
- **Prompt Engineering Ético Estricto:** Es vital que configuren el *System Prompt* del modelo con instrucciones rigurosas de gobernanza. El agente debe utilizar únicamente un lenguaje consultivo de *"posible fraude"* o *"requiere revisión humana"*, garantizando por completo que la IA jamás emita una acusación directa de fraude. Esto les asegurará la nota máxima en la dimensión de Ética.
- **Habilitar las Consultas Agénticas (Crucial para la Demo):** No dejen al LLM solo como un generador de resúmenes pasivos. Conéctenlo a la base de datos (utilizando herramientas de consulta SQL o ejecutando funciones Python controladas) para responder las preguntas exactas exigidas en el reto. El jurado solicitará en vivo consultas como: *"¿Qué proveedores concentran el 80% de las alertas rojas?"* o *"¿Cuáles son los 10*

sinistros con mayor riesgo de posible fraude?". Si su agente responde esto consultando los datos en tiempo real, tendrán el hackathon en el bolsillo.

4. Interfaz de Usuario y Repositorio

- **Dashboard en Streamlit:** Desarrollen una aplicación visual ágil en `app/main.py`. Debe incluir tres secciones principales alineadas con los casos de uso del reto: la bandeja de siniestros priorizados por score (CU-03), la sección de detalle con la explicación automática del riesgo (CU-04) y el chat interactivo para consultas en lenguaje natural (CU-05).
- **Estructura de Repositorio Estricta:** Organicen las carpetas de su GitHub exactamente como lo solicita la guía del reto (`src/ingestion/`, `src/rules/`, `src/models/`, `src/ai_agent/`, `src/explainability/`, `app/`). No olviden incluir el archivo `requirements.txt`, un `.env.example` limpio y los 5 archivos markdown obligatorios de documentación en la carpeta correspondiente (`arquitectura.md`, `modelo_datos.md`, `reglas_negocio.md`, `uso_ia.md` y `limitaciones.md`).

El Toque Diferenciador para Asegurar el Primer Lugar:

La rúbrica premia de forma excepcional la capacidad de **"detectar patrones complejos como redes de relación"**. Aprovechando que la arquitectura incluye la tabla de *Beneficiarios/Proveedores*, agreguen en una pestaña del dashboard de Streamlit un pequeño grafo interactivo (usando librerías como `networkx` o `pyvis`) que muestre visualmente las conexiones entre talleres, peritos y asegurados recurrentes. Esto demostrará visualmente que su solución cruza múltiples variables de forma inteligente.

¡Tienen el mapa de ruta perfecto! Distribuyan las tareas en su equipo entre `backend/datos`, `analítica/IA`, y `frontend/documentación`, y ejecuten con confianza. ¡A por el primer lugar!